

Projeto Estrutura Dados

Arquitetura e Organização do Projeto

Jogo Investigativo — Estrutura de Dados (2º Período ADS)

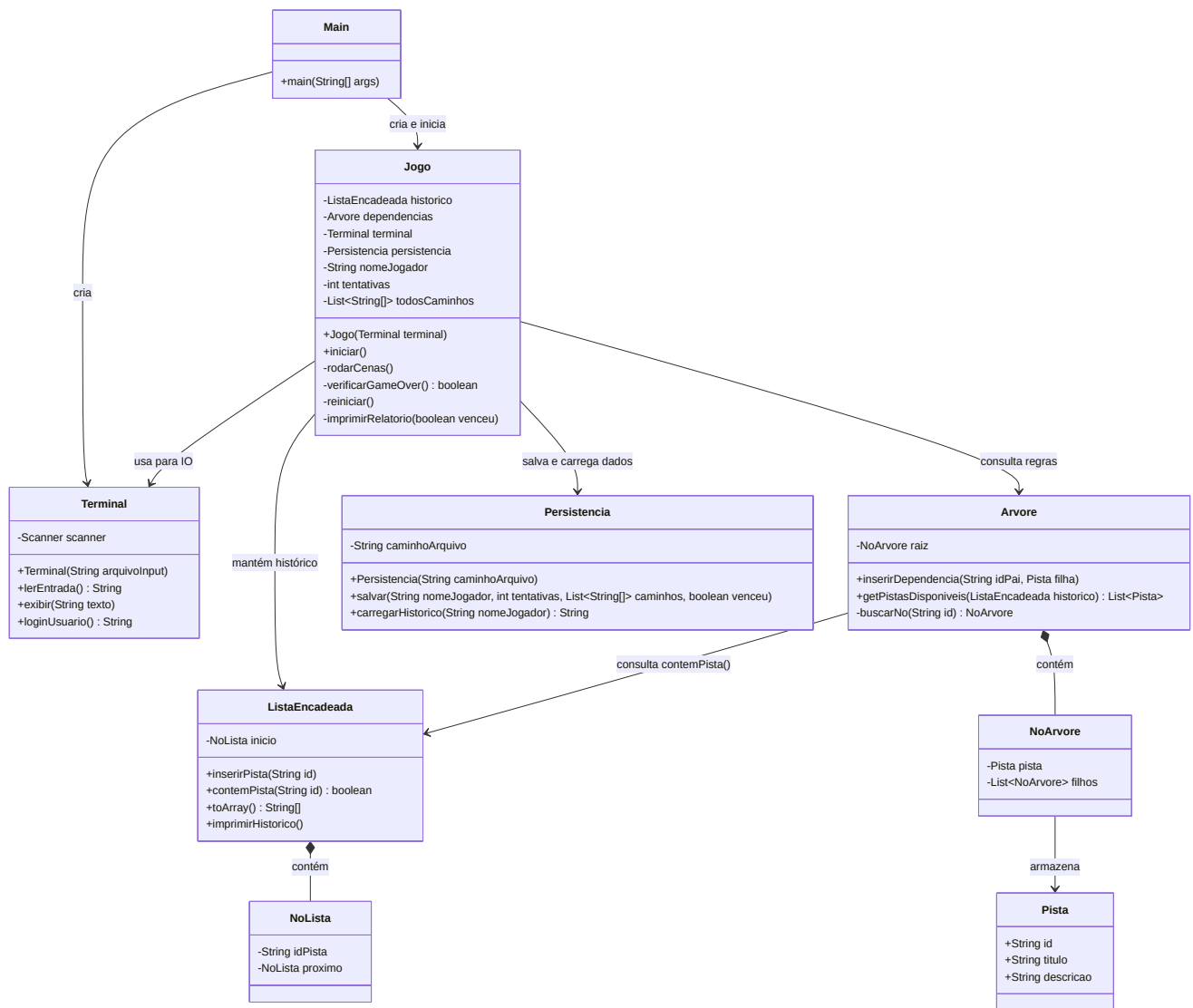
Este documento descreve **o que cada classe faz, como elas se conversam e quem implementa cada parte**. Leia antes de começar a codificar.

1. Visão Geral do Sistema

O jogo roda no terminal. O jogador faz login, lê a narrativa de uma cena e escolhe qual pista investigar. O sistema registra cada escolha em uma **Lista Encadeada** (o histórico) e consulta uma **Árvore** (o gabarito lógico) para decidir o que acontece a seguir. Se errar, o jogo salva o caminho percorrido, reinicia e permite nova tentativa. Ao final, exibe um **relatório com todos os caminhos** da sessão e persiste os dados em arquivo.

2. Diagrama de Classes

Cada caixa é uma classe Java. As setas mostram qual classe usa qual. As bolinhas cheias (* -) indicam que uma classe contém a outra.



3. O Que Cada Classe Faz

Main

Ponto de entrada do programa. Cria um `Terminal` e um `Jogo`, passa um para o outro e chama `iniciar()`. Não contém nenhuma lógica de jogo.

Se o jogo for chamado com um arquivo de script:

```
java Main test_inputs/vitoria.txt
```

A `Main` lê o argumento e passa para o `Terminal`, que vai usá-lo no lugar do teclado.

Terminal

É o único ponto de contato entre o jogo e o mundo externo — para exibir texto e para ler o que o jogador digita. Nenhuma outra classe usa `Scanner` ou `System.out.println` diretamente.

Por que existe essa classe?

O `Terminal` resolve um problema prático: **testar o jogo exige digitar todas as escolhas manualmente a cada execução**. Ao centralizar toda leitura aqui, é possível substituir o teclado por um arquivo de texto sem mudar nada no resto do jogo.

Como funciona em uma cena

A cada cena, o `Jogo` exibe o menu de pistas e chama `terminal.lerEntrada()` para saber a escolha do jogador. O `Terminal` devolve uma `String` — e não importa de onde ela veio:

Modo teclado:

```
[Menu: "1. Faca    2. Janela Arrombada"]
Jogo chama → terminal.lerEntrada()
Jogador digita "1" e aperta Enter
Terminal devolve → "1"
```

Modo arquivo:

```
[Menu: "1. Faca    2. Janela Arrombada"]
Jogo chama → terminal.lerEntrada()
Terminal lê a próxima linha do arquivo .txt → "1"
Terminal devolve → "1"
```

O comportamento do jogo é **idêntico nos dois casos**.

Como montar o arquivo de script

Cada linha do arquivo é uma entrada que o jogador daria, na ordem em que o jogo vai pedi-las:

```
Andre
1
2
1
```

Linha 1 → nome do jogador (login)

Linha 2 → escolha na Cena 1

Linha 3 → escolha na Cena 2

Linha 4 → escolha na Cena 3

Métodos

Método	O que faz
<code>Terminal(String arquivoInput)</code>	Se <code>arquivoInput</code> for <code>null</code> , lê do teclado. Se for um caminho de arquivo, lê dali.
<code>lerEntrada()</code>	Retorna a próxima linha — do teclado ou do arquivo, de forma transparente.
<code>exibir(String texto)</code>	Imprime texto na tela.
<code>loginUsuario()</code>	Pede o nome do jogador e retorna como <code>String</code> .

Jogo

O coração do sistema. Controla o fluxo: login, cenas, validação, relatório e reinício. Não faz IO diretamente — sempre delega ao `Terminal`.

Como os caminhos anteriores são preservados: A cada `reiniciar()`, o jogo salva um snapshot do histórico atual em `todosCaminhos` antes de limpar a `ListaEncadeada`. No final da sessão, `imprimirRelatorio()` percorre essa lista e mostra todos os caminhos tentados.

Método	O que faz
<code>iniciar()</code>	Faz o login, consulta o histórico do jogador via <code>persistencia.carregarHistorico()</code> , monta a <code>Arvore</code> com o gabarito e entra no loop de cenas.
<code>rodarCenas()</code>	Para cada cena, chama <code>getPistasDisponiveis()</code> na <code>Árvore</code> , exibe o menu, lê a escolha e insere na <code>ListaEncadeada</code> . Ao final de cada cena, checa se houve Game Over.
<code>verificarGameOver()</code>	Verifica se a última pista inserida não tem filhos na <code>Árvore</code> (beco sem saída). Retorna <code>true</code> se o jogo terminou.
<code>reiniciar()</code>	Salva <code>historico.toArray()</code> em <code>todosCaminhos</code> , incrementa <code>tentativas</code> , limpa o histórico (<code>historico = new ListaEncadeada()</code>) e volta ao início de <code>rodarCenas()</code> .
<code>imprimirRelatorio(boolean venceu)</code>	Exibe todos os caminhos tentados e o resultado final (ver seção 4). Depois chama <code>persistencia.salvar()</code> .

Pista

Objeto de dados puro. Representa uma evidência. Não tem lógica — apenas armazena informação.

```
Pista faca = new Pista("faca", "Faca de cozinha", "Uma faca com manchas escuras na lâmina.");
```

Ter a classe `Pista` permite que a `Arvore` carregue o texto da evidência junto com o nó. O `Jogo` lê a descrição diretamente do nó e exibe ao jogador — sem `if/switch` gigante.

ListaEncadeada e NoLista

Estrutura de dados nº 1. Registra cronologicamente todas as pistas coletadas pelo jogador — é o "histórico da investigação".

Cada escolha do jogador é adicionada ao final da lista com `inserirPista(id)`. Ao exibir o relatório, `imprimirHistorico()` mostra:

```
[faca] -> [impressao_digital] -> [suspeito_capturado] -> FIM
```

Por que `contemPista(String id)` existe: A `Arvore` precisa saber se o jogador já coletou determinada pista antes de liberar as próximas. Em vez de receber a lista inteira e navegar por ela mesma, a `Arvore` simplesmente pergunta: *"você contém essa pista?"* e recebe um `boolean`. As duas estruturas permanecem independentes.

Por que `toArray()` existe: Quando o jogo reinicia, a `ListaEncadeada` é zerada. Para não perder o caminho anterior, o `Jogo` chama `toArray()` e guarda o snapshot em `todosCaminhos` antes de limpar.

Método	O que faz
<code>inserirPista(String id)</code>	Cria um <code>NoLista</code> com o <code>id</code> e adiciona no fim da lista.
<code>contemPista(String id)</code>	Percorre a lista e retorna <code>true</code> se o <code>id</code> foi encontrado.
<code>toArray()</code>	Retorna todas as pistas coletadas como um array de Strings. Usado pelo <code>Jogo</code> para salvar o caminho antes de reiniciar.
<code>imprimirHistorico()</code>	Imprime todas as pistas no formato <code>A -> B -> C -> FIM</code> .

Arvore e NoArvore

Estrutura de dados nº 2. É o gabarito do caso. Define quais pistas existem, quais dependem de quais, e qual é o caminho correto.

Cada `NoArvore` guarda uma `Pista` e uma lista de filhos. Os filhos de um nó só ficam acessíveis se a pista do nó pai já foi coletada. Exemplo de como o gabarito é estruturado:

```
Raiz (nó inicial, sem pista)
├─ NÓ: "faca"
│   └─ NÓ: "impressao_digital"    ← caminho correto → leva à vitória
│       └─ NÓ: "marca_de_bota"    ← caminho errado → Game Over
└─ NÓ: "janela_arrombada"
    └─ NÓ: "fibra_de_tecido"      ← caminho errado → Game Over
```

Como o `getPistasDisponiveis()` funciona: A `Arvore` percorre todos os seus nós e, para cada um, pergunta à `ListaEncadeada` se o nó pai já foi coletado. Se sim, a pista daquele nó entra na lista de disponíveis. O `Jogo` usa essa lista para montar o menu de cada cena dinamicamente.

Método	O que faz
<code>inserirDependencia(String idPai, Pista filha)</code>	Localiza o nó <code>idPai</code> via <code>buscarNo()</code> e adiciona filha como filho. É assim que o gabarito é montado em <code>iniciar()</code> .
<code>getPistasDisponiveis(ListaEncadeada historico)</code>	Retorna todas as pistas cujo nó pai já está no histórico. Encapsula a lógica de acesso — não existe método <code>verificarAcesso</code> separado.
<code>buscarNo(String id)</code> (<i>privado</i>)	Busca recursivamente o nó com o <code>id</code> informado. Usado internamente por <code>inserirDependencia</code> .

Persistencia

Responsável por gravar e ler os dados da sessão em arquivo. É a única classe que toca no sistema de arquivos.

O arquivo de persistência (ex: `dados/partidas.txt`) é um registro cumulativo de todas as sessões jogadas. Cada sessão grava um bloco de texto no final do arquivo.

Por que isso fecha o requisito de login: Ao fazer login, o `Jogo` chama `persistencia.carregarHistorico(nomeJogador)`. Se o jogador já jogou antes, o sistema

exibe suas últimas partidas antes de começar — dando ao login um propósito real além de pedir um nome.

Método	O que faz
<code>Persistencia(String caminhoArquivo)</code>	Recebe o caminho do arquivo onde os dados serão gravados/lidos (ex: "dados/partidas.txt"). Cria o arquivo e a pasta se não existirem.
<code>salvar(String nomeJogador, int tentativas, List<String[]> caminhos, boolean venceu)</code>	Grava o resultado da sessão no arquivo: nome, data/hora, número de tentativas, todos os caminhos percorridos e o resultado.
<code>carregarHistorico(String nomeJogador)</code>	Lê o arquivo e retorna o histórico de partidas daquele jogador como texto formatado. Retorna uma string vazia se for a primeira vez.

4. O Relatório Final

O método `imprimirRelatorio(boolean venceu)` no `Jogo` exibe um bloco formatado com **todos os caminhos tentados na sessão**, não apenas o último. Em seguida, persiste os dados chamando `persistencia.salvar()`.

Exemplo: jogador errou na 1ª tentativa e acertou na 2ª

```
=====
                        RELATÓRIO DE INVESTIGAÇÃO
=====
Detetive   : Andre
Tentativas: 2
Data/Hora  : 2026-06-28 14:32

--- Tentativa 1 (FALHOU) ---
[faca] -> [marca_de_bota] -> FIM
x "marca_de_bota" levou a um beco sem saída.

--- Tentativa 2 (SUCESSO) ---
[faca] -> [impressao_digital] -> [suspeito_capturado] -> FIM

Resultado : CASO RESOLVIDO – Suspeito identificado!
=====
```

O relatório usa exclusivamente os dados já armazenados nas estruturas:

- **Nome e tentativas** → atributos do `Jogo`

- **Todos os caminhos** → `List<String[]> todosCaminhos` , populada a cada `reiniciar()`
- **Caminho final** → `historico.toArray()` da última rodada
- **Data/hora** → `LocalDateTime.now()` do Java padrão

Após exibir, o `Jogo` chama `persistencia.salvar()` para gravar tudo em arquivo.

5. Divisão de Tarefas — Final de Semana

Módulo 1 — Estruturas de Dados (Desenvolvedor A)

Arquivos: `Pista.java` , `NoLista.java` , `ListaEncadeada.java` , `NoArvore.java` , `Arvore.java`

- Implementar `Pista` (só atributos e construtor).
- Implementar `NoLista` e `ListaEncadeada` com todos os métodos listados na seção 3.
- Implementar `NoArvore` e `Arvore` , incluindo o método **privado** `buscarNo()` .
- Testar localmente antes de subir no repositório: inserir pistas na lista e confirmar que `contemPista()` e `toArray()` funcionam; montar uma árvore pequena e confirmar que `getPistasDisponiveis()` retorna os nós corretos.

Prazo: Sábado de manhã.

Módulo 2 — Motor do Jogo e Narrativa (Desenvolvedor B)

Arquivos: `Jogo.java`

- Escrever o roteiro: textos das 3 cenas e descrições das pistas (vão para os objetos `Pista`).
- Dentro de `iniciar()` , montar a `Arvore` com `inserirDependencia()` — este é o gabarito do caso.
- Implementar `rodarCenas()` , `verificarGameOver()` , `reiniciar()` (com o snapshot em `todosCaminhos`) e `imprimirRelatorio()` .
- Não usar `Scanner` ou `System.out.println` diretamente — sempre `terminal.lerEntrada()` e `terminal.exibir()` .

Prazo: Sábado de manhã (narrativa e estrutura do loop). Integração no Sábado à tarde.

Módulo 3 — Interface, IO, Scripts e Persistência (Desenvolvedor C)

Arquivos: `Terminal.java`, `Main.java`, `Persistencia.java`,
`test_inputs/vitoria.txt`, `test_inputs/derrota.txt`

- Implementar `Terminal` conforme descrito na seção 3.
- Implementar `Persistencia`: criar o arquivo de dados se não existir, gravar e ler sessões por nome de jogador.
- Implementar `Main`: ler `args[0]` (se existir), criar `Terminal`, `Persistencia` e `Jogo`, chamar `iniciar()`.
- Criar os arquivos de script: `vitoria.txt` e `derrota.txt`.

Prazo: Sábado de manhã — os scripts de teste e a `Persistencia` devem estar prontos antes da integração.

Integração e Testes (Todos Juntos)

Sábado à tarde:

1. Dev A entrega as estruturas no repositório.
2. Dev B e C integram na `Main` e no `Jogo`.
3. Rodar `java Main test_inputs/vitoria.txt` e `java Main test_inputs/derrota.txt` para validar o fluxo completo, incluindo a geração do arquivo de dados.
4. Verificar se o arquivo de persistência foi criado e se `carregarHistorico()` retorna o histórico corretamente no próximo login.

Domingo:

- Garantir estabilidade do `reiniciar()` — jogar, perder, reiniciar, jogar de novo, conferir o relatório com os dois caminhos.
- Polir a apresentação visual no terminal (separadores, arte ASCII, mensagens de Game Over).
- Ensaiar a apresentação usando o modo automatizado.